

WordPress Domain Migration Guide

Comprehensive Standard Operating Procedure (SOP) for cPanel, macOS, and Flatsome Theme

This document establishes the definitive, error-free workflow for updating a WordPress site's primary domain name. It mitigates risk associated with hardcoded files, server environment restrictions, and specialized serialized data structural integrity (specifically focusing on **Flatsome** and **Page Builders**).

1. The Core Hazard: Serialized Data Explained

WordPress heavily utilizes PHP **Serialized Data** within the database (especially in the `wp_options` table) to store arrays and objects as simple strings. These strings explicitly enforce character counts. For example: `s:14:"https://old.com";`.

If a global Linux `sed` command or basic SQL find-and-replace is executed directly on a raw database dump without decoding the string structure, the character metadata will mismatch (e.g., swapping a 14-character URL with a 15-character URL). This completely invalidates the record, corrupting complex theme configurations, page layouts, and plugin options.

2. Master 5-Step Server Migration Workflow

When server-side Terminal or SSH access is fully available on the cPanel hosting environment, execute the steps in this specific chronological order to maximize synchronization:

Step 1: Modify Hosting Main Domain & DNS

- Update the primary main domain within the cPanel **Domains** section.
- Ensure the **Document Root** points to the exact same folder containing the current source code (typically `public_html`).
- Update external DNS system A-records and provision an **SSL Certificate (Let's Encrypt)** for the new domain name immediately.

Step 2: Hardcode Safe Mode in wp-config.php

To bypass any automatic application-level redirect loop mechanisms immediately following the hosting switch, insert the following definitions directly below the opening `<?php` line:

```
define('WP_HOME', 'https://newdomain.com');
define('WP_SITEURL', 'https://newdomain.com');
```

Step 3: Replace Hardcoded Values in Source Files

Before executing the database migration, sanitize static text assets, inline theme elements, custom code configurations, or asset references hardcoded directly in code files (`wp-content/`). Navigate to your root directory and run:

```
find . -type f \( -name "*.php" -o -name "*.css" -o -name "*.js" -o -name "*.json" \) | xargs sed -i 's/olddomain.com/newdomain.com/g'
```

Step 4: Database Search & Replace via WP-CLI

Execute the official WP-CLI binary replacement protocol. This safely un-serializes, updates, and re-serializes all nested strings correctly:

```
wp search-replace 'https://olddomain.com' 'https://newdomain.com' --all-tables
```

Note: Once successful, you may safely comment out or delete the static constants added in Step 2 from your `wp-config.php`.

Step 5: Flush Layout Assets & Cache Systems

- **Elementor:** Navigate to `Elementor` → `Tools` → `Replace URL`. Input old and new strings. Then under `General`, click **Regenerate Files & Data**.
- **Divi:** Navigate to `Divi` → `Theme Options` → `Builder` → `Advanced` → Click **Clear** on Static CSS File Generation.
- **Permalinks:** Navigate to `Settings` → `Permalinks` and click **Save Changes** twice to rewrite the server `.htaccess` rules.

3. The Flatsome Theme Specific Adjustments

Flatsome is uniquely sensitive to migrations because its **UX Builder** saves drag-and-drop structural elements within text shortcodes in `wp_posts`, and stores the comprehensive layout configuration options under a single database entry string (`theme_mods_flatsome`).

Mandatory Flatsome Post-Migration Actions:

1. Log into the newly updated WordPress English Dashboard.
2. Go to **Flatsome** → **Theme Options** → Click the blue **Save All** button. This explicitly forces a database refresh and recalculates structural string sizes for the theme options array.
3. Go to **Flatsome** → **Advanced** → **Maintenance** → Click **Clear All Cache** to strip legacy compiled theme layouts.

4. macOS Offline Workflow (No Server-Side WP-CLI)

If your hosting lacks terminal controls, download a backup archive zip file and process it entirely offline using native macOS terminal controls.

A. Source File Replacement on BSD/macOS Unix

macOS `sed` requires an explicit fallback definition string parameter. Use an empty string `''` right after the `-i` attribute to execute inline replacements safely:

```
cd ~/Downloads/your_extracted_site_folder
find . -type f \( -name "*.php" -o -name "*.css" -o -name "*.js" -o -name "*.json" \)
-exec sed -i '' 's/olddomain.com/newdomain.com/g' {} +
```

B. Safe Database Manipulation Locally

To perform a serialized-safe database switch on your flat `.sql` backup file locally on macOS without setting up local servers, fetch the standalone WP-CLI archive file wrapper:

```
curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar
php wp-cli.phar search-replace "https://olddomain.com" "https://newdomain.com" --
unbuffered --all-tables < backup_old_db.sql > migrated_new_db.sql
```

*Alternative approach: Use local ecosystem wrappers like **LocalWP** or **MAMP**, import your downloaded files, use the integrated site shell to run native WP-CLI alterations, then export the finalized results back to the live server environment.*